Uppsala University

Scientific Computing for Data Analysis
VT2025

1TD352

Miniproject 1: Genetic oscillator

Authors:
Subash K C and Jakob Hjelm Redin

April 11, 2025

1 Introduction

Our environment contains many cyclic processes such as, the daily alternation of light and dark. Many organisms have developed molecular mechanisms known as "circadian clocks", to adapt to such changes. In this miniproject, we study and reproduce parts of the results and figures in a research article "Mechanisms of noise-resistance in genetic oscillators" by Vilar, Kueh, Barkai and Leible (2002), using both deterministic approach and stochastic simulations. The model consists of two key proteins: an activator (A) and a repressor (R), along with their corresponding genes (D_A and D_R). The activated forms of these genes (D'_A and D'_R) occur when the activator protein binds to them. We will explore how the concentrations of two proteins—an activator (A) and a repressor (R)—change over time, and compare how noise affects the behavior of the system. We will also discuss the difference between the deterministic and the stochastic solution in our particular case and the choice of mathematical models, i.e. a deterministic vs stochastic model.

2 Approach And Methods

We will reproduced the results from `Figure 2` and `Figure 5` of the original article1 using both deterministic and stochastic simulations over a 400-hour period. Those figures describe the dynamics of two proteins: the activator (A) and the repressor (R), over a simulation period of 400 hours. The model parameters and initial values were given in the caption of `Figure 1` of the article1 are as follows:

Fig. 1. Biochemical network of the circadian oscillator model. D_A' and D_A denote the number of activator genes with and without A bound to its promoter respectively; similarly, D_R' and D_R refer to the repressor promoter; M_A and M_R denote mRNA of A and R; A and R correspond to the activator and repressor proteins; and C corresponds to the inactivated complex formed by A and R. The constants α and α' denote the basal and activated rates of transcription, β the rates of translation, δ the rates of spontaneous degradation, γ the rates of binding of A to other components, and θ denotes the rates of unbinding of A from those components. Except if otherwise stated, in this paper we have assumed the following values for the reaction rates: $\alpha_A = 50$ h^{-1}, $\alpha_A' = 500$ h^{-1}, $\alpha_R = 0.01$ h^{-1}, $\alpha_R' = 50$ h^{-1}, $\beta_A = 50$ h^{-1}, $\beta_R = 5$ h^{-1}, $\delta_{MA} = 10$ h^{-1}, $\delta_{MR} = 0.5$ h^{-1}, $\delta_A = 1$ h^{-1}, $\delta_R = 0.2$ h^{-1}, $\gamma_A = 1$ mol^{-1} hr^{-1}, $\gamma_R = 1$ mol^{-1} hr^{-1}, $\gamma_C = 2$ mol^{-1} hr^{-1}, $\theta_A = 50$ h^{-1}, $\theta_R = 100$ h^{-1}, where mol means number of molecules. The initial conditions are $D_A = D_R = 1$ mol, $D_A' = D_R' = M_A = M_R = A = R = C = 0$, which require that the cell has a single copy of the activator and repressor genes: $D_A + D_A' = 1$ mol and $D_R + D_R' = 1$ mol. The cellular volume is assumed to be the unity so that concentrations and number of molecules are equivalent. Notice that we assume that the complex breaks into R because of the degradation of A and, therefore, the parameter δ_A appears twice in the model.

Figure 1: Intial values from fig.1 caption

2.1 Deterministic Model Implementation

The deterministic dynamics of the model are described by the system the following ordinary differential equations (ODEs):

$$\begin{cases} \frac{dD_A}{dt} = \theta_A D'_A - \gamma_A D_A A \\ \frac{dD_R}{dt} = \theta_R D'_R - \gamma_R D_R A \\ \frac{dD'_A}{dt} = \gamma_A D_A A - \theta_A D'_A \\ \frac{dD'_R}{dt} = \gamma_R D_R A - \theta_R D'_R \\ \frac{dM_A}{dt} = \alpha_A D_A + \alpha'_A D'_A - \delta_{MA} M_A \end{cases} \tag{1}$$

$$\begin{cases} \frac{dA}{dt} = \beta_A M_A + \theta_A D'_A + \theta_R D'_R - A(\gamma_A D_A + \gamma_R D_R + \gamma_C R + \delta_A) + \delta_A C \\ \frac{dM_R}{dt} = \alpha_R D_R + \alpha'_R D'_R - \delta_{MR} M_R \\ \frac{dR}{dt} = \beta_R M_R - \gamma_C AR + \delta_A C - \delta_R R \\ \frac{dC}{dt} = \gamma_C AR - \delta_A C \end{cases}$$

We solved these equations numerically using the `solve_ivp` function from SciPy's `integrate` module, employing the Backward Differentiation Formula (BDF) method since the ODE is stiff. This way we reproduce the `Figure`2, which is basically the same as the results in $Fig2a$, and $Fig2b$ of the article1.

2.2 Stochastic Model Implementation

For the stochastic implementation, we used Stochastic Simulation Algorithm (SSA) from lab session to model the system as a set of 16 elementary reactions:

$$\begin{cases} 1.\ A + R \xrightarrow{\gamma_C} C \\ 2.\ A \xrightarrow{\delta_A} \\ 3.\ C \xrightarrow{\delta_A} R \\ 4.\ R \xrightarrow{\delta_R} \\ 5.\ D_A + A \xrightarrow{\gamma_A} D'_A \\ 6.\ D_R + A \xrightarrow{\gamma_R} D'_R \end{cases} \quad \begin{cases} 7.\ D'_A \xrightarrow{\theta_A} A + D_A \\ 8.\ D_A \xrightarrow{\alpha_A} D_A + M_A \\ 9.\ D'_A \xrightarrow{\alpha'_A} D'_A + M_A \\ 10.\ M_A \xrightarrow{\delta_{MA}} \\ 11.\ M_A \xrightarrow{\beta_A} A + M_A \\ 12.\ D'_R \xrightarrow{\theta_R} A + D_R \end{cases} \tag{2}$$

$$\begin{cases} 13.\ D_R \xrightarrow{\alpha_R} D_R + M_R \\ 14.\ D'_R \xrightarrow{\alpha'_R} D'_R + M_R \\ 15.\ M_R \xrightarrow{\delta_{MR}} \\ 16.\ M_R \xrightarrow{\beta_R} M_R + R \end{cases}$$

The approach consists of the following steps:

1. **Random Number Generation**: Random numbers are drawn from exponential and discrete distributions to simulate the waiting times between reactions and to select the next reaction, respectively.

2. **Propensity Functions**: The rates (propensities) of each reaction are calculated based on the current state of the system and reaction-specific parameters. For stochastic model, the propensity function is defined as follows:

$$
\begin{aligned}
w_0 &= \gamma_C \cdot A \cdot R \\
w_1 &= \delta_A \cdot A \\
w_2 &= \delta_A \cdot C \\
w_3 &= \delta_R \cdot R \\
w_4 &= \gamma_A \cdot DA \cdot A \\
w_5 &= \gamma_R \cdot DR \cdot A \\
w_6 &= \theta_A \cdot DA' \\
w_7 &= \alpha_A \cdot DA \\
w_8 &= \alpha'_A \cdot DA' \\
w_9 &= \delta_{MA} \cdot MA \\
w_{10} &= \beta_A \cdot MA \\
w_{11} &= \theta_R \cdot DR' \\
w_{12} &= \alpha_R \cdot DR \\
w_{13} &= \alpha'_R \cdot DR' \\
w_{14} &= \delta_{MR} \cdot MR \\
w_{15} &= \beta_R \cdot MR
\end{aligned}
\tag{3}
$$

   where the rates are parameter-dependent, such as γ_A, δ_A, and α_A.

3. **State Updates**: After selecting a reaction, the state of the system is updated by applying the corresponding changes from the state-change matrix. The state-change matrix describes how each reaction affects

the system state. For our model, the matrix is as follows:

$$\text{StateChangeMat} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 \\ -1 & 0 & 1 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & -1 & 0 & 1 & 0 & 0 & -1 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \tag{4}$$

4. **Time Advancement**: The simulation proceeds in discrete time steps, with the time between reactions determined by the exponential waiting times.

The simulation runs until a specified final time is reached, and the trajectories of key species (Activator and Repressor) are plotted to visualize their stochastic dynamics.

2.3 Deterministic Model vs Stochastic Model

In order to see the differences in Deterministic Model and Stochastic Model we reproduce the `Figure 5` from the article1. We use the same method as above for both Deterministic and Stochastic model, but with modified degradation rate of the repressor protein from: $\delta_R = 0.2$ to $\delta_R = 0.05$. Then we plot the concentration of R over time.

3 Result

The Deterministic model yielded regular, sustained oscillations of A and R proteins over time. The period and amplitude were consistent with those shown in the article as we can see in `Figure`2.

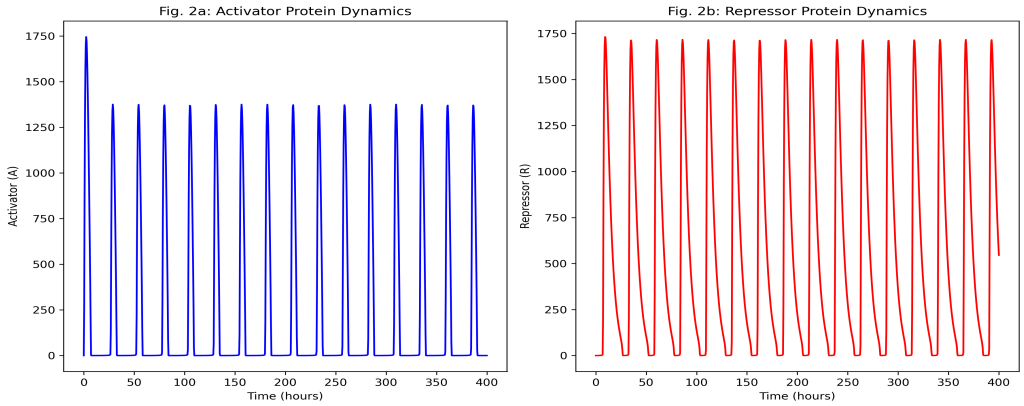

Figure 2: Deterministic model to see how both proteins A (activator) and R (repressor) vary over 400 hours

Meanwhile stochastic model introduced variability in amplitude and period between cycles. Despite this noise, the core oscillatory behavior persisted, which we can see in `Figure3` . But when we run the stochastic simulation (using the SSA) multiple times, we observe that the overall oscillatory behavior of the system remains intact, but the exact trajectory of each simulation varies. The period and amplitude of the protein concentrations (A and R) fluctuate from one run to another. This is expected, since the stochastic model accounts for the random timing of individual reactions.

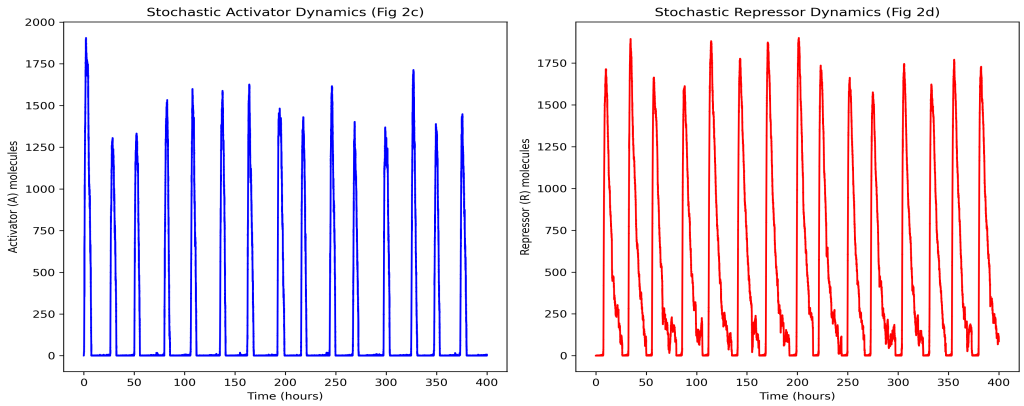

Figure 3: Stochastic model to see how both proteins A (activator) and R (repressor) vary over 400 hours

With altered parameters ($\delta_R = 0.05h^{-1}$), the deterministic model reached a steady state. However, stochastic simulations under the same parameters still exhibited sustained oscillations, highlighting noise-induced dynamics, which can be seen in `Figure`4.

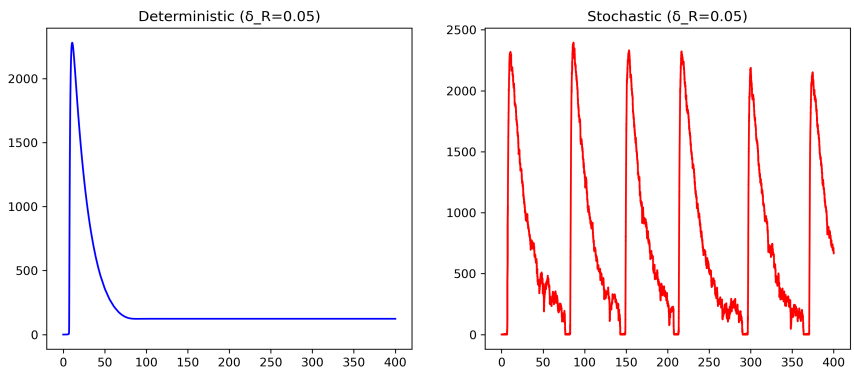

Figure 4: Deterministic vs Stochastic model to see how proteins R (repressor) vary over 400 hours with $\delta_R = 0.05$

4 Discussion

The choice between using a deterministic or a stochastic model depends on the biological system being studied and the specific question at hand. In general, deterministic models (like ODEs) assume continuous concentrations and average behavior. They are best suited for systems with large numbers of molecules and well-mixed conditions, where the law of large numbers smooths out random fluctuations. Additionally deterministic approach is computationally efficient than stochastic model. However, cells are small and crowded, and many important biochemical reactions involve very low numbers of molecules—sometimes just a few copies of a gene, mRNA, or protein. In such cases, stochastic effects become significant, and deterministic models may fail to capture critical dynamics. As stated in the article:

"The preceding equations would be strictly valid in a well-stirred macroscopic reactor. At the cellular level, a more realistic approach has to consider the intrinsic stochasticity of chemical reactions."

This highlights that stochastic models, such as those based on Gillespie's

SSA, are more realistic at the molecular scale. Our simulations support this and highlights key differences between deterministic and stochastic modeling approaches for the genetic oscillator system. In the standard case `Figure`2 and 3, both models show similar oscillatory behavior, indicating that noise does not completely disrupt the dynamics. However, when we changed the degradation rate of the repressor protein from ($\delta_R = 0.2h^{-1}$) to ($\delta_R = 0.05h^{-1}$), the deterministic model predict a stable steady state, while the stochastic model continued to oscillate(See `Figure`4). This is a clear example of noise-induced oscillations, where random fluctuations drive behavior that wouldn't occur in the noise-free limit.

In conclusion, deterministic models are useful when we're interested in the average behavior of a system—things are smooth, predictable, and there's little randomness involved. Stochastic models, on the other hand, are better when randomness plays a big role—like in systems with low molecule counts or in gene regulation. They help us capture unpredictable behaviors and noise-driven effects that deterministic models can miss. Even through the deterministic model is computationally faster than stochastic models, in practice it's often helpful to use both approaches: deterministic models to understand the overall trends and structure, and stochastic models to explore how randomness might change the outcome.

5 References

1. Vilar, J. M. G., Kueh, H. Y., Barkai, N., & Leibler, S. (2002). Mechanisms of noise-resistance in genetic oscillators. *Proceedings of the National Academy of Sciences*, 99(9), 5988–5992.

2. Lectures notes

3. Lab 1 `Lab1_P4.ipynb`

Appendix

```python
# -*- coding: utf-8 -*-
"""Miniproject1.ipynb
"""



"""Question 1. Deterministic"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from google.colab import files

# Model parameters (from Fig. 1 caption)
alpha_A = 50.0    # h^-1
alpha_A_prime = 500.0  # h^-1
alpha_R = 0.01    # h^-1
alpha_R_prime = 50.0   # h^-1
beta_A = 50.0     # h^-1
beta_R = 5.0      # h^-1
delta_MA = 10.0   # h^-1
delta_MR = 0.5    # h^-1
delta_A = 1.0     # h^-1
delta_R = 0.2     # h^-1
gamma_A = 1.0     # molecule^-1 h^-1
gamma_R = 1.0     # molecule^-1 h^-1
gamma_C = 2.0     # molecule^-1 h^-1
theta_A = 50.0    # h^-1
theta_R = 100.0   # h^-1

# Initial conditions (from Fig. 1 caption)
D_A = 1.0    # molecule
D_R = 1.0    # molecule
D_A_prime = 0.0  # molecule
D_R_prime = 0.0  # molecule
M_A = 0.0    # molecule
M_R = 0.0    # molecule
A = 0.0      # molecule
R = 0.0      # molecule
C = 0.0      # molecule

# Time span (0 to 400 hours)
t_span = (0, 400)
```

```python
t_eval = np.linspace(0, 400, 1000)  # Time points for evaluation

# the ODE system of equation in  (Eq. [1]) modeified from lab session
def model(t, y, delta_R=0.2):
    D_A, D_R, D_A_prime, D_R_prime, M_A, A, M_R, R, C = y

    # Activator gene dynamics
    dD_A_dt = theta_A * D_A_prime - gamma_A * D_A * A
    dD_A_prime_dt = gamma_A * D_A * A - theta_A * D_A_prime

    # Repressor gene dynamics
    dD_R_dt = theta_R * D_R_prime - gamma_R * D_R * A
    dD_R_prime_dt = gamma_R * D_R * A - theta_R * D_R_prime

    # mRNA dynamics
    dM_A_dt = alpha_A * D_A + alpha_A_prime * D_A_prime - delta_MA * M_A
    dM_R_dt = alpha_R * D_R + alpha_R_prime * D_R_prime - delta_MR * M_R

    # Protein dynamics
    dA_dt = (beta_A * M_A + theta_A * D_A_prime + theta_R * D_R_prime
             - A * (gamma_A * D_A + gamma_R * D_R + gamma_C * R + delta_A))
    dR_dt = beta_R * M_R - gamma_C * A * R + delta_A * C - delta_R * R
    dC_dt = gamma_C * A * R - delta_A * C

    return [dD_A_dt, dD_R_dt, dD_A_prime_dt, dD_R_prime_dt,
            dM_A_dt, dA_dt, dM_R_dt, dR_dt, dC_dt]

# Initial state vector
y0 = [D_A, D_R, D_A_prime, D_R_prime, M_A, A, M_R, R, C]

# Solve the ODEs
sol = solve_ivp(model, t_span, y0, t_eval=t_eval, method='BDF')

# Extract results
t = sol.t
A = sol.y[5]  # Activator protein
R = sol.y[7]  # Repressor protein
```

```python

# Plot the results like in (Fig. 2a and 2b)
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.plot(t, A, 'b')
plt.xlabel('Time (hours)')
plt.ylabel('Activator (A)')
plt.title('Fig. 2a: Activator Protein Dynamics')

plt.subplot(1, 2, 2)
plt.plot(t, R, 'r')
plt.xlabel('Time (hours)')
plt.ylabel('Repressor (R)')
plt.title('Fig. 2b: Repressor Protein Dynamics')

plt.tight_layout()
plt.savefig('Fig1.png', dpi=300)
plt.show()

"""Question 2. Stochastic)"""

import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict


# Generate N random numbers from exponential distribution
def RandExp(lam, N):
    U = np.random.rand(N)
    X = -1/lam * np.log(1-U)
    return X


#  Generate N random numbers from discrete distribution
#   with sorted state vector x and corresponding probability vector p
def RandDisct(x, p, N):
    cdf = np.cumsum(p)
    U = np.random.rand(N)
    idx = np.searchsorted(cdf, U)
    return x[idx]


# Initial conditions (molecule counts)
```

```python
Initial = [1, 1, 0, 0, 0, 0, 0, 0, 0]  # [DA, DR, DA_prime, DR_prime, MA, MR, A, R, C]

# State-change matrix (16 reactions x 9 species)
StateChangeMat = np.array([
    # A+ R C  (0)
    [0, 0, 0, 0, 0, 0, -1, -1, 1],
    #   A     (1)
    [0, 0, 0, 0, 0, 0, -1, 0, 0],
    #  C R  (2)
    [0, 0, 0, 0, 0, 0, 0, 1, -1],
    #   R     (3)
    [0, 0, 0, 0, 0, 0, 0, -1, 0],
    # DA+  A  DA_prime   (4)
    [-1, 0, 1, 0, 0, 0, -1, 0, 0],
    # DR+  A  DR_prime   (5)
    [0, -1, 0, 1, 0, 0, -1, 0, 0],
    # DA_primeA+DA (6)
    [1, 0, -1, 0, 0, 0, 1, 0, 0],
    #  D A DA +MA (7)
    [0, 0, 0, 0, 1, 0, 0, 0, 0],
    # DA_primeDA_prime+MA (8)
    [0, 0, 0, 0, 1, 0, 0, 0, 0],
    #   M  A    (9)
    [0, 0, 0, 0, -1, 0, 0, 0, 0],
    #  M A A +MA (10)
    [0, 0, 0, 0, 0, 0, 1, 0, 0],
    # DR_primeA+DR (11)
    [0, 1, 0, -1, 0, 0, 1, 0, 0],
    #  D R DR +MR (12)
    [0, 0, 0, 0, 0, 1, 0, 0, 0],
    # DR_primeDR_prime+MR (13)
    [0, 0, 0, 0, 0, 1, 0, 0, 0],
    #   M  R    (14)
    [0, 0, 0, 0, 0, -1, 0, 0, 0],
    #  M R MR +R (15)
    [0, 0, 0, 0, 0, 0, 0, 1, 0]
])

# Propensity function for the genetic oscillator
def PropensityFunc(State, ReactNo):
    DA, DR, DA_prime, DR_prime, MA, MR, A, R, C = State
    w = np.zeros(ReactNo)

    w[0] = gamma_C * A * R
```

```
    w[1] = delta_A * A
    w[2] = delta_A * C
    w[3] = delta_R * R
    w[4] = gamma_A * DA * A
    w[5] = gamma_R * DR * A
    w[6] = theta_A * DA_prime
    w[7] = alpha_A * DA
    w[8] = alpha_A_prime * DA_prime
    w[9] = delta_MA * MA
    w[10] = beta_A * MA
    w[11] = theta_R * DR_prime
    w[12] = alpha_R * DR
    w[13] = alpha_R_prime * DR_prime
    w[14] = delta_MR * MR
    w[15] = beta_R * MR

    return w



# SSA implementation (modified from labsession)
def SSA(Initial, StateChangeMat, FinalTime):
    AllTimes = []
    AllStates = []
    t = 0
    State = np.array(Initial)

    AllTimes.append(t)
    AllStates.append(State.copy())

    while t < FinalTime:
        w = PropensityFunc(State, len(StateChangeMat))
        a = np.sum(w)

        if a == 0:
            break

        tau = RandExp(a, 1)[0]
        t += tau

        if t > FinalTime:
            break

        which = RandDisct(np.arange(len(StateChangeMat)), w/a, 1)[0]
```

```
        State += StateChangeMat[which]

        AllTimes.append(t)
        AllStates.append(State.copy())

    return np.array(AllTimes), np.array(AllStates)

# Run simulation
FinalTime = 400
Times, States = SSA(Initial, StateChangeMat, FinalTime)

# Extract A and R trajectories
A_traj = States[:, 6]
R_traj = States[:, 7]

# Plot results
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.plot(Times, A_traj, 'b')
plt.xlabel('Time (hours)')
plt.ylabel('Activator (A) molecules')
plt.title('Stochastic Activator Dynamics (Fig 2c)')

plt.subplot(1, 2, 2)
plt.plot(Times, R_traj, 'r')
plt.xlabel('Time (hours)')
plt.ylabel('Repressor (R) molecules')
plt.title('Stochastic Repressor Dynamics (Fig 2d)')

plt.tight_layout()
plt.savefig('Fig2.png', dpi=300)
plt.show()

"""Question 3:"""

# now solving ode by deterministic using delta_R= 0.05 instead of 0.2
sol1 = solve_ivp(lambda t,y:model(t,y,delta_R=0.05),
                     [0,400], y0, method='BDF')
```

```python


# Solving ode by stochatic method with delta_R= 0.05 insteda of 0.2
def PropensityFunc1(State, ReactNo , delta_R = 0.05):
    DA, DR, DA_prime, DR_prime, MA, MR, A, R, C = State
    w = np.zeros(ReactNo)

    w[0] = gamma_C * A * R
    w[1] = delta_A * A
    w[2] = delta_A * C
    w[3] = delta_R * R
    w[4] = gamma_A * DA * A
    w[5] = gamma_R * DR * A
    w[6] = theta_A * DA_prime
    w[7] = alpha_A * DA
    w[8] = alpha_A_prime * DA_prime
    w[9] = delta_MA * MA
    w[10] = beta_A * MA
    w[11] = theta_R * DR_prime
    w[12] = alpha_R * DR
    w[13] = alpha_R_prime * DR_prime
    w[14] = delta_MR * MR
    w[15] = beta_R * MR

    return w



# SSA implementation (modified from labsession)
def SSA1(Initial, StateChangeMat, FinalTime, delta_R = 0.05):
    AllTimes = []
    AllStates = []
    t = 0
    State = np.array(Initial)

    AllTimes.append(t)
    AllStates.append(State.copy())

    while t < FinalTime:
        w = PropensityFunc1(State, len(StateChangeMat), delta_R= 0.05)
        a = np.sum(w)

        if a == 0:
```

```
            break

        tau = RandExp(a, 1)[0]
        t += tau

        if t > FinalTime:
            break

        which = RandDisct(np.arange(len(StateChangeMat)), w/a, 1)[0]
        State += StateChangeMat[which]

        AllTimes.append(t)
        AllStates.append(State.copy())

    return np.array(AllTimes), np.array(AllStates)

times1, states1 = SSA1(Initial, StateChangeMat, FinalTime, delta_R = 0.05)

# Extract R trajectories using new delta_R
R_traj1 = states1[:, 7]


# Plot comparison
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(sol1.t, sol1.y[7], 'b')
plt.title('Deterministic ( _R =0.05)')


plt.subplot(1,2,2)
plt.plot(times1,R_traj1 , 'r')
plt.title('Stochastic ( _R =0.05)')
plt.savefig('Fig3.png', dpi=300)
plt.show()

"""Question 4:"""
```

Listing 1: Python Code for Miniproject